Data Types in the Kernel

Kernel Data Types

- Should compile with the following flags
  - `-Wall`
  - `-Wextra`
  - `-Wconversion`
  - `-Wshadow`
  - `-Wstrict-prototypes`
    - Warn if a function is declared or defined without specifying the argument types.
- Three main classes
  - Standard C types (e.g., `int`)
  - Explicitly sized types (e.g., `u32`)
  - Types for specific kernel objects (e.g., `pid_t`)

Use of Standard C Types

- Normal C types are not the same size on all architectures
- Try **`misc-progs/datasize`**

```
% misc-progs/datasize
arch Size: char short int long ptr long-long u8 u16 u32 u64
i686          1    2    4    4    4      8     1   2   4   8
```

- Try **`misc-modules/kdatasize`** to see kernel versions

Use of Standard C Types

- 64-bit platforms have different data type representations

arch	Size:	char	short	int	long	ptr	long-long	u8	u16	u32	u64
i386		1	2	4	4	4	8	1	2	4	8
alpha		1	2	4	8	8	8	1	2	4	8
armv4l		1	2	4	4	4	8	1	2	4	8
ia64		1	2	4	8	8	8	1	2	4	8
m68k		1	2	4	4	4	8	1	2	4	8
mips		1	2	4	4	4	8	1	2	4	8
ppc		1	2	4	4	4	8	1	2	4	8
sparc		1	2	4	4	4	8	1	2	4	8
sparc64		1	2	4	4	4	8	1	2	4	8
x86_64		1	2	4	8	8	8	1	2	4	8

Use of Standard C Types

- Knowing that pointers and long integers have the same size
  - Using **`unsigned long`** for kernel addresses prevents unintended pointer dereferencing

Assigning an Explicit Size to Data Items

- See **<asm/types.h>**
  - **u8; /* unsigned byte (8-bits) */**
  - **u16; /* unsigned word (16-bits) */**
  - **u32; /* unsigned 32-bit value */**
  - **u64; /* unsigned 64-bit value */**
- If a user-space program needs to use these types, use __ prefix (e.g., **__u8**)
- For example, , a driver needs to exchange binary structures with a program running in user space by means of ioctl, the header files should declare 32-bit fields in the structures as __u32.

Interface-Specific Types

- Datatypes in the kernel have their own typedef statements, preventing portability problems.
- *Interface-specific type*: type defined by a library to provide an interface to specific data structure (e.g., **`pid_t`**)

Interface-Specific Types

- Many `_t` types are defined in `<linux/types.h>`
- **`Don't follow the convention, compiler throws warning.`**
  - Problematic in **`printk`** statements
  - One solution is to cast the value to the biggest possible type (e.g., **`unsigned long long`**)
    - Avoids stack corruption
    - Avoids warning messages
    - Will not output improper value

Other Portability Issues

- In addition to data typing, few software issues to keep in mind, when writing a driver.
- *General rule* - Be suspicious of explicit constant values
- Most values are parameterized with preprocessor macros

Timer Intervals

- Do not assume 1000 jiffies per second
  - Scale times using **`HZ`** (number of interrupts per second)
    - For example, check against a timeout of half a second, compare the elapsed time against **`HZ/2`**
    - Number of jiffies corresponding to **`msec`** second is always **`msec*HZ/1000`**

Page Size

- Memory page is **`PAGE_SIZE`** bytes, not 4KB
  - Can vary from 4KB to 64KB
  - PAGE_SIZE and PAGE_SHIFT macros
  - **`PAGE_SHIFT`** contains the number of bits to shift an address to get its page number. The number is 12 or greater for pages that are 4KB or larger.
  - See **`<asm/page.h>`**
  - User-space program can use **`getpagesize`** library function

Page Size

- Example
  - To allocate 16KB
    - Should not specify an order of 2 to **`__get_free_pages`**
    - Use **`get_order`**

```
#include <asm/page.h>

int order = get_order(16*1024);
buf = __get_free_pages(GFP_KERNEL, order);
```

Byte Order

- PC stores multibyte values low-byte first (little-endian) and some high level platforms (big-endian).
- Not care for byte ordering in the data it manipulates.
- Use predefined macro - `<asm/byteorder.h>`
  - `<linux/byteorder/big_endian.h>`
  - `<linux/byteorder/little_endian.h>`

Byte Order

- Examples
  - `u32 cpu_to_le32(u32);`
    - `cpu` = internal CPU representation
    - `le` = little endian
  - `u64 be64_to_cpu(u64);`
    - `be` = big endian
  - `U16 cpu_to_le16p(u16);`
    - `p` = pointer
    - Converts value pointed to by p

Data Alignment

- How to read a 4-byte value stored at an address that is not a multiple of 4 bytes?
  - x86 permits this kind of access
  - Not all architectures permit it (e.g., ARM)
    - Raises exceptions

Data Alignment Example

```
char *data = ...;
unsigned long l = *(unsigned long *)data;
```

- Treats the pointer to a char as a pointer to an unsigned long, which might result in the 32- or 64-bit unsigned long value being loaded from an address that is not a multiple of 4 or 8, respectively.

Data Alignment

- Use the following typeless macros
  - `#include <asm/unaligned.h>`
  - `get_unaligned(ptr);`
  - `put_unaligned(val, ptr);`

Data Alignment

- Another issue is the portability of data structures
  - Compiler rearranges structure fields to be aligned according to platform-specific conventions
  - Automatically add padding to make things aligned
    - May no longer match the intended format

Data Alignment

- For example, consider the following structure on a 32-bit machine

```
struct animal_struct {
        char dog; /* 1 byte */
        unsigned long cat; /* 4 bytes */
        unsigned short pig; /* 2 bytes */
        char fox; /* 1 byte */
};
```

Data Alignment

- Structure not laid out like that in memory
  - Natural alignment of structure's members is inefficient
- Instead, complier creates padding

```
struct animal_struct {
        char dog; /* 1 byte */
        u8 __pad0[3]; /* 3 bytes */
        unsigned long cat; /* 4 bytes */
        unsigned short pig; /* 2 bytes */
        char fox; /* 1 byte */
        u8 __pad1; /* 1 byte */
};
```

Data Alignment

- You can often rearrange the order of members in a structure to obviate the need for padding

```
struct animal_struct {
        unsigned long cat; /* 4 bytes */
        unsigned short pig; /* 2 bytes */
        char dog; /* 1 byte */
        char fox; /* 1 byte */
};
```

Data Alignment

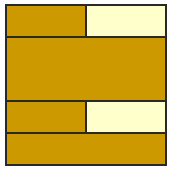
- Another option is to tell the compiler to pack the data structure with no fillers added
- Example: **<linux/edd.h>**

```
struct {
  u16 id;
  u64 lun;
  u16 reserved1;
  u32 reserved2;
} __attribute__ ((packed)) scsi;
```

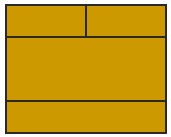
Without **__attribute__ ((packed))**, **lun** would be preceded by 2-6 bytes of fillers

Data Alignment

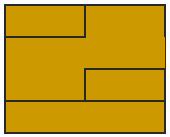
- No compiler optimizations



- Some compiler optimizations



- __attribute__ ((packed))



Pointers and Error Values

- Functions that return pointers cannot report negative error values
  - Return **`NULL`** on failure
- Some kernel interfaces encode error code in a pointer value
  - Cannot be compared against **`NULL`**
  - To use this feature, include **`<linux/err.h>`**

Pointers and Error Values

- To return an error, use
  - `void *ERR_PTR(long error);`
- To test whether a returned pointer is an error code, use
  - `long IS_ERR(const void *ptr);`
- To access the error code, use
  - `long PTR_ERR(const void *ptr);`